SIMATS ENGINEERING

ASSESSMENT TOOL 1

Name:S.Naveen Kumar
Reg no:192425176
Marks:88

Item	Details
Course Name	INTERNET PROGRAMMING
Course Code	CSA4304
Course Outcome Covered	CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP mechanisms for web content delivery. (L2)
Assessment Tool	Application-Based Questions

Question 1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate HTML5 semantic elements missing in the structure.
2. Modify the structure to make it standards-compliant.
3. Interpret the HTTP request generated during form submission.
4. Analyze the HTTP response cycle from server to browser.
5. Suggest improvements for usability and accessibility.

Question 2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
```

```
  </div>
 </body>
</html>
```

Questions:

1. Identify improper nesting and missing closing tags.
2. Analyze how different browsers may interpret this structure.
3. Identify missing semantic elements (e.g., header, section).
4. Provide a corrected W3C-compliant HTML structure.
5. Suggest techniques to ensure cross-browser compatibility.

Question 3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
 <input type="text" name="username">
 <input type="password" name="password">
 <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the form configuration.
2. Analyze why the HTTP request is not triggered.
3. Interpret the role of GET method in this scenario.
4. Propose a corrected implementation.
5. Suggest improvements for secure data transmission.

Code of Conduct

I S. Naveen Kumar -192425176 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S.Naveen.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

1.Online Symposium Portal

1.1 Missing HTML5 semantic elements

Semantic elements describe the **meaning** of content, not just its appearance. They help browsers, search engines and assistive technologies such as screen readers understand the page. The given snippet is only a bare form fragment, and it is missing the following:

Missing element	Why it is needed
<!DOCTYPE html>, <html lang>, <head>, <title>, <meta charset>	Without a DOCTYPE the browser enters quirks mode. lang lets screen readers pick the right pronunciation. charset prevents garbled characters and <title> names the page in tabs and search results.
<header>	Holds the symposium name, logo and introductory content.
<nav>	Marks the navigation links (Home, Events, Schedule, Contact).
<main>	Identifies the unique primary content of the page. Only one <main> is allowed per page.
<section> (with a heading)	Groups the registration area as a thematic block, for example "Student Registration".
<fieldset> and <legend>	Groups related form controls and gives the group an accessible name.
<label>	Associates visible text with each input. Without it a screen reader announces only "edit text".
<footer>	Holds copyright, contact and organiser information.

Also missing in the form itself: a type="submit" on the button, id attributes to link labels to inputs, and the required attribute for mandatory fields.

1.2 Standards-compliant structure

The corrected page adds the DOCTYPE, language, character set, viewport, semantic landmarks, labels and validation attributes:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Symposium Registration</title>
</head>
```

```
<body>
  <header>
    <h1>National Technical Symposium 2026</h1>
    <nav aria-label="Main navigation">
      <ul>
        <li><a href="/">Home</a></li>
        <li><a href="/events">Events</a></li>
        <li><a href="/contact">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main>
    <section aria-labelledby="reg-heading">
      <h2 id="reg-heading">Student Registration</h2>
      <form action="/register" method="POST">
        <fieldset>
          <legend>Participant details</legend>

          <label for="name">Full name</label>
          <input type="text" id="name" name="name"
                 required autocomplete="name"
                 placeholder="e.g., Priya Sharma">

          <label for="email">Email address</label>
          <input type="email" id="email" name="email"
                 required autocomplete="email"
                 aria-describedby="email-help">
          <small id="email-help">We will send your confirmation here.</small>

          <button type="submit">Register</button>
        </fieldset>
      </form>
    </section>
  </main>

  <footer>
    <p>&amp;copy; 2026 College Symposium Committee</p>
  </footer>
</body>
</html>
```

Result: the page validates against the W3C HTML5 validator, uses correct nesting, and exposes landmarks (header, nav, main, footer) that assistive technologies can navigate directly.

1.3 Interpreting the HTTP request

When the student clicks **Submit**, the browser collects every control that has a `name` attribute, encodes the values as `application/x-www-form-urlencoded` (the default `enctype`) and sends an HTTP request. Because `method="POST"`, the data travels in the **message body** rather than in the URL. For example, with name "Priya Sharma" and email "priya@college.edu":

```
POST /register HTTP/1.1
Host: symposium.college.edu
User-Agent: Mozilla/5.0 (...)
Accept: text/html,application/xhtml+xml
```

```
Content-Type: application/x-www-form-urlencoded
Content-Length: 43
Origin: https://symposium.college.edu
Referer: https://symposium.college.edu/
Cookie: session=abc123

name=Priya+Sharma&email=priya%40college.edu
```

Part of the request	Interpretation
Request line	POST is the method, /register is the resource taken from the form action, and HTTP/1.1 is the protocol version.
Host header	Names the server (needed for virtual hosting).
Content-Type	Tells the server how to parse the body: key=value pairs joined by &.
Content-Length	Size of the body in bytes (43 in this example).
Origin / Referer	Show where the request came from and are used for CSRF defence.
Cookie	Carries session information because HTTP itself is stateless.
Blank line	Separates headers from the body.
Body	The submitted data. Spaces become + and special characters are percent-encoded (@ becomes %40). The keys name and email come from the input name attributes. An input without a name is **not sent**.

1.4 HTTP response cycle from server to browser

The full cycle runs through these stages:

1. **Connection set-up:** the browser resolves the domain through DNS, opens a TCP connection to the server and, for HTTPS, completes the TLS handshake.
2. **Request sent:** the POST request shown above is transmitted.
3. **Server processing:** the web server routes /register to the application code. The code validates the inputs, checks for duplicates, saves the record in the database and may send a confirmation email.
4. **Response built:** the server creates a status line, headers and, optionally, a body.
5. **Response returned:** the response travels back over the same connection.
6. **Browser handling:** the browser reads the status code, processes headers (for example Set-Cookie), parses the HTML into the DOM, requests linked CSS, JS and images, and renders the page.

A typical well-designed server answers a successful POST with a redirect (the Post/Redirect/Get pattern), which prevents duplicate registrations if the user refreshes the page:

```
HTTP/1.1 303 See Other
```

```
Location: /thank-you
Set-Cookie: session=abc123; HttpOnly; Secure

--- browser automatically issues: GET /thank-you ---

HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Content-Length: 1520

<!DOCTYPE html>
<html lang="en"> ... Registration successful ... </html>
```

Status code	Meaning	Typical situation in this portal
200 OK	Request succeeded	Confirmation page returned directly.
303 See Other	Redirect to another URL using GET	Redirect to the thank-you page after saving.
400 Bad Request	Malformed or invalid data	Email format rejected by the server.
409 Conflict	Conflicts with current state	Email already registered.
500 Internal Server Error	Server-side failure	Database is down.

Key point: HTTP is stateless and follows a request-response model, so every response is independent. Continuity such as login state is maintained using cookies or tokens.

1.5 Improvements for usability and accessibility

Usability

- Use the correct input types (email, tel) so mobile devices show the appropriate keyboard.
- Add required, pattern and maxlength for instant client-side validation, and always repeat validation on the server.
- Show clear, specific error messages next to the field, such as "Enter a valid email like name@college.edu".
- Use autocomplete values and sensible placeholders to reduce typing effort.
- Display a confirmation page or message after submission and send a confirmation email.

- Use a responsive layout with the viewport meta tag and large, touch-friendly buttons.
- Mark mandatory fields clearly and keep the form as short as possible.

Accessibility (WCAG)

- Associate every input with a <label for>; never rely on placeholder text alone.
- Use aria-describedby for hints and aria-live or role="alert" regions for error messages.
- Ensure full keyboard operability with a logical tab order and a visible focus outline.
- Maintain a colour contrast ratio of at least 4.5:1 and do not use colour as the only way to show an error.
- Declare the page language with lang and use a proper heading hierarchy (h1, then h2).
- Provide a "skip to main content" link and use landmark elements for easy screen-reader navigation.

Security and reliability

- Serve the portal over HTTPS and add a CSRF token to the POST form.
- Add CAPTCHA or rate limiting to block bots, and sanitise all input on the server to prevent injection attacks.

2.Cross-Browser Compatibility Issue

2.1 Improper nesting and missing closing tags

Problem	Explanation
<h1> is never closed	There is no </h1>. The heading is left open, so the following element ends up inside it.
is never closed	There is no . HTML5 permits omitting in some cases, but XHTML and good practice require it, and it makes the structure ambiguous.
Improper nesting	A (block-level flow content) appears inside an unclosed <h1>. Headings may only contain phrasing content, so the must not be a child of the <h1>.
Missing <!DOCTYPE html>	Triggers quirks mode, which is a major source of layout differences.
Missing lang, <meta charset> and viewport meta	Leads to encoding issues, accessibility problems and poor rendering on mobile devices.

2.2 How different browsers may interpret the structure

HTML is a forgiving language. Browsers never reject invalid markup; they **repair** it using error-recovery rules. When markup is invalid, the repair can differ, and that produces inconsistent layout.

- **Quirks mode:** with no DOCTYPE, browsers emulate old behaviour. The box model, default font sizes inside tables, line heights and image alignment can all differ from standards mode, and the differences vary between browsers.
- **Error recovery:** modern browsers follow the HTML5 parsing algorithm. In this snippet the start tag does not close the open <h1>, so the paragraph "About us" is parsed as a **child of the heading** and inherits the heading's large, bold style. The then forces both elements closed. The DOM therefore becomes div > h1 > (text + p), not the intended div > h1 and div > p siblings.
- **Older or non-compliant browsers** (for example IE6 to IE8 or legacy mobile browsers) used their own recovery rules. They might close the heading early, ignore the paragraph margin, or apply different default margins, so the same page looks different.

- **Different default style sheets:** each browser has its own default margins, padding and font sizes for h1, p and body. Invalid structure makes these differences more visible.
- **Accessibility tree:** a screen reader would announce the whole paragraph as part of the heading, which damages the document outline.

Conclusion: invalid markup makes the result depend on each browser's repair behaviour. Valid markup removes this ambiguity.

2.3 Missing semantic elements

- <header> for the site title and introductory content.
- <nav> for the primary menu (Home, Services, Contact).
- <main> as the single wrapper for the central content.
- <section> (or <article>) for the "About us" content, with its own heading such as <h2>.
- <footer> for contact details and copyright.
- The generic carries no meaning and should be replaced wherever a semantic element fits.

2.4 Corrected W3C-compliant structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Company</title>
</head>
<body>
  <header>
    <h1>Welcome</h1>
    <nav aria-label="Main navigation">
      <a href="/">Home</a>
      <a href="/about">About</a>
      <a href="/contact">Contact</a>
    </nav>
  </header>

  <main>
    <section id="about">
      <h2>About us</h2>
      <p>We are a technology company delivering reliable software solutions.</p>
    </section>
  </main>

  <footer>
    <p>&copy; 2026 Company Name. All rights reserved.</p>
  </footer>
</body>
</html>
```

What changed: every element is properly opened, closed and nested; the heading contains only text; the paragraph is a sibling inside a <section>; the DOCTYPE forces standards mode; and semantic landmarks replace the bare .

2.5 Techniques to ensure cross-browser compatibility

Technique	How it helps
Valid, standards-based markup	Always start with <!DOCTYPE html> and validate with the W3C Markup Validator so every browser builds the same DOM.
CSS reset / normalize.css	Removes differences in default margins, padding and font sizes between browsers.
Feature detection, not browser sniffing	Use @supports in CSS, or Modernizr / if ("fetch" in window) in JavaScript, to test for features rather than for browser names.
Progressive enhancement and graceful degradation	Build a working baseline in plain HTML and CSS, then add advanced features for browsers that support them.
Vendor prefixes (Autoprefixer)	Automatically adds -webkit-, -moz- prefixes for CSS properties that need them.
Polyfills and transpilers (Babel)	Supply missing JavaScript and CSS features in older browsers.
Responsive design	Use the viewport meta tag, relative units (%, rem, em), flexbox/grid and media queries so layout adapts to any screen.
Check support data	Consult caniuse.com and MDN compatibility tables before using a new feature.
Cross-browser testing	Test on Chrome, Firefox, Safari and Edge, on mobile devices, and with tools such as BrowserStack or DevTools device mode.
Accessible fallbacks	Provide alt text, fallback fonts and fallback content (<noscript>) so the page stays usable.

3. Form Submission Failure

3.1 Issues in the form configuration

No.	Issue	Explanation
1	<button type="button">	A type="button" button has **no default action**. It does not submit the form. This is the main cause of the failure.
2	method="GET" for a login form	Credentials would be appended to the URL as a query string, exposing the password in the address bar, browser history, server logs and the Referer header.
3	No JavaScript handler	Nothing in the page (no onclick or submit listener) triggers the submission manually.
4	Missing <label> elements and ids	Hurts accessibility and usability.
5	No required attribute or validation	Empty username or password can be sent.
6	No autocomplete hints	Password managers and browsers cannot fill the fields reliably.
7	No evidence of HTTPS or CSRF protection	Credentials could be intercepted or forged across sites.

3.2 Why the HTTP request is not triggered

A form is submitted when a **submit event** fires. Inside a <form>, the browser fires it only when:

- the user clicks a <button type="submit"> (the default type for a button in a form), an <input type="submit"> or an <input type="image">; or
- the user presses Enter inside a text field and the form has a submit button; or
- JavaScript calls form.submit() or form.requestSubmit().

Here the button is explicitly declared type="button". This overrides the default submit behaviour, so clicking it does nothing. No submit event is generated, the browser never builds the request, and no HTTP traffic leaves the client. The action and method attributes are therefore never used. (Pressing Enter in a field also fails because the form has no submit button.)

```
Click Login (type="button")
      |
      v
No submit event -> no form data collected -> no HTTP request sent
```

3.3 Role of the GET method in this scenario

If the button were fixed but method="GET" was kept, the browser would send:

```
GET /submit?username=arun&password=MySecret123 HTTP/1.1
Host: portal.example.com
```

- **Purpose of GET:** it is designed to **retrieve** resources. It is safe and idempotent, and the data goes in the URL query string.
- **Security problem:** the password is visible in the address bar, saved in browser history and bookmarks, written to server and proxy logs, and leaked through the Referer header to other sites.
- **Other limits:** URL length is limited (roughly 2,000 characters in practice), and responses may be cached.
- **Conclusion:** GET is appropriate for search or filter forms. A login form changes server state and carries sensitive data, so it must use **POST**, which sends data in the request body.

3.4 Corrected implementation

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login</title>
</head>
<body>
  <main>
    <h1>Login</h1>
    <form action="/submit" method="POST">
      <label for="username">Username</label>
      <input type="text" id="username" name="username"
           required autocomplete="username">

      <label for="password">Password</label>
      <input type="password" id="password" name="password"
           required minlength="8" autocomplete="current-password">

      <button type="submit">Login</button>
    </form>
  </main>
</body>
</html>
```

Changes made: type="button" became type="submit"; GET became POST; labels, ids, required, minlength and autocomplete were added; and the page was given a valid structure. The request now becomes:

```
POST /submit HTTP/1.1
Host: portal.example.com
Content-Type: application/x-www-form-urlencoded

username=arun&password=MySecret123
```

The credentials are now in the body, not the URL. The body is still readable on the network unless HTTPS is used, which is covered next.

3.5 Improvements for secure data transmission

Measure	Description
HTTPS / TLS everywhere	Encrypts data in transit so eavesdroppers cannot read credentials. Make the action URL HTTPS and redirect all HTTP traffic to HTTPS.
HSTS header	Strict-Transport-Security forces browsers to always use HTTPS for the site.
Use POST, never GET, for sensitive data	Keeps credentials out of URLs, history and logs.
Hash passwords on the server	Store passwords with a slow salted hash such as bcrypt, scrypt or Argon2. Never store plain text.
CSRF protection	Include a hidden anti-CSRF token in the form and validate it on the server; use SameSite cookies.
Secure session cookies	Set HttpOnly, Secure and SameSite flags so scripts cannot read the session cookie.
Server-side validation and sanitisation	Never trust client checks. Use prepared statements to stop SQL injection and escape output to stop XSS.

Measure	Description
Brute-force protection	Rate limiting, account lockout or CAPTCHA, and generic error messages ("invalid username or password").
Multi-factor authentication	Adds a second proof of identity, such as an OTP, beyond the password.
Avoid logging secrets and use CSP	Do not write passwords to logs. A Content Security Policy reduces script injection risk.

Criteria	Weigh tage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	23	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	20	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	18	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	17	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand